

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09-2

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

Nº de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 03 de Noviembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Descripción

En esta parte de la práctica se realizaron dos nuevas animaciones utilizando transformaciones geométricas, funciones matemáticas y algoritmos dependientes del tiempo.

En el primer ejercicio se retomó el modelo del Holocron, separando su cuerpo principal y sus ocho piezas exteriores. La animación busca reproducir el comportamiento mostrado en el GIF de referencia: el Holocron se eleva, flota y gira mientras sus esquinas se separan, rotan y posteriormente regresan a su posición original.

En el segundo ejercicio se agregó un cohete espacial que realiza un despegue automático. Su trayectoria se obtiene mediante ecuaciones que modifican progresivamente su altura y desplazamiento lateral. También se calcula su inclinación durante el vuelo y se agregaron efectos de fuego y humo para representar el funcionamiento del motor.

Desarrollo

Ejercicio 1. Animación del Holocron

Para realizar este ejercicio se utilizó el Holocron dividido en diferentes componentes. El modelo está formado por un cuerpo principal y ocho piezas independientes correspondientes a las cuatro esquinas superiores y cuatro inferiores.

Las ocho piezas se cargan por separado, lo que permite aplicarles transformaciones individuales durante la animación.

El movimiento general del Holocron se controla mediante la variable:

double tiempoHolocron = 0.0;

Durante cada ciclo de renderizado esta variable aumenta utilizando el tiempo transcurrido.

tiempoHolocron += deltaTimeRover;

Elevación, flotación y giro

La elevación inicial se obtiene mediante:

const GLfloat subida =1.0 - std::exp(-0.8 * tiempoHolocron);

Este valor comienza cerca de cero y aumenta progresivamente.

Posteriormente se calcula la altura:

const GLfloat altura =2.5f +6.0f * subida +0.45f * subida *std::sin(1.2 * tiempoHolocron);

Además del ascenso, se utiliza una función seno para producir una pequeña oscilación vertical. Esto permite que el Holocron parezca permanecer flotando después de alcanzar su altura.

También se genera un giro constante alrededor del eje Y:

Además del ascenso, se utiliza una función seno para producir una pequeña oscilación vertical. Esto permite que el Holocron parezca permanecer flotando después de alcanzar su altura.

También se genera un giro constante alrededor del eje Y:

const GLfloat giro =glm::radians(std::fmod(20.0 * tiempoHolocron, 360.0));

y una pequeña inclinación:

const GLfloat inclinacion =glm::radians(3.0f) * subida * std::sin(0.7 * tiempoHolocron);

Estas transformaciones se combinan dentro de una matriz padre

```
1066        // flotar y girar el holocron
1067        {
1068            tiempoHolocron += deltaTimeRover;
1069            const GLfloat subida = static_cast<GLfloat>(1.0-std::exp(-0.8*tiempoHolocron));
1070            const GLfloat altura = 2.5f+6.0f*subida+
1071                0.45f*subida*static_cast<GLfloat>(std::sin(1.2*tiempoHolocron));
1072            const GLfloat giro = glm::radians(static_cast<GLfloat>(std::fmod(20.0*tiempoHolocron,360.0)));
1073            const GLfloat inclinacion = glm::radians(3.0f)*subida*
1074                static_cast<GLfloat>(std::sin(0.7*tiempoHolocron));
1075            glm::mat4 holocronPadre = glm::translate(glm::mat4(1),glm::vec3(-35.0f,altura,40.0f));
1076            holocronPadre = glm::rotate(holocronPadre,giro,glm::vec3(0,1,0));
1077            holocronPadre = glm::rotate(holocronPadre,inclinacion,glm::vec3(0,0,1));
1078            holocronPadre = glm::scale(holocronPadre,glm::vec3(0.8f));
```

Figura 1. Cálculo de la elevación, flotación y rotación automática del Holocron en función del tiempo.

Ciclo de apertura y cierre

La animación de las esquinas tiene una duración total de 15 segundos. Durante los primeros tres segundos el Holocron permanece cerrado.

Después comienza la parte de la animación correspondiente a la apertura, giro y cierre de las piezas.

Durante los primeros tres segundos el Holocron permanece cerrado.

Después comienza la parte de la animación correspondiente a la apertura, giro y cierre de las piezas.

Para evitar movimientos bruscos se utiliza una función de suavizado y después se calcula el ángulo de las fases de las esquinas y la separación respecto al cuerpo, así las piezas se separan progresivamente, alcanzan un punto máximo de apertura y después regresan al cuerpo.

```
1100            // 3 segundos cerrado y 12 segundos para abrir, girar y cerrar
1101            const double tiempoCicloEsquinas = std::fmod(tiempoHolocron,15.0);
1102            GLfloat avanceEsquinas = 0.0f;
1103            if (tiempoCicloEsquinas > 3.0)
1104                avanceEsquinas = static_cast<GLfloat>((tiempoCicloEsquinas-3.0)/12.0);
1105            // empieza y termina el giro suavemente
1106            const GLfloat avanceSuave = avanceEsquinas*avanceEsquinas*(3.0f-2.0f*avanceEsquinas);
1107            const GLfloat faseEsquinas = glm::radians(360.0f*avanceSuave);
1108            const GLfloat senoApertura = std::sin(faseEsquinas*0.5f);
1109            const GLfloat apertura = 3.0f*senoApertura*senoApertura;
1110            const glm::vec3 origenEsquinas(0.00407529f,0.04012322f,0.01084948f);
1111            static const glm::vec3 direcciones[8] = {
1112                {-1,1,-1}, {-1,1,1}, {1,1,1}, {1,1,-1},
1113                {-1,-1,-1}, {-1,-1,1}, {1,-1,1}, {1,-1,-1}
1114            };
1115            static const glm::vec3 ejes[8] = {
1116                {1,1,1}, {1,1,-1}, {-1,1,-1}, {-1,1,1},
1117                {1,-1,1}, {1,-1,-1}, {-1,-1,-1}, {-1,-1,1}
1118            };
```

Figura 2. Cálculo y aplicación de la apertura, rotación y cierre progresivo de las ocho piezas exteriores del Holocron.

Movimiento de las ocho piezas

Cada una de las ocho esquinas tiene una dirección, centro y eje de rotación propio.

Las cuatro superiores giran en un sentido y las cuatro inferiores en sentido contrario:

const GLfloat angulo =(i < 4)? faseEsquinas: -faseEsquinas;

Posteriormente cada pieza se desplaza hacia afuera y se rota alrededor de su centro.

Esto permite que la animación no dependa únicamente de mover todo el Holocron, sino que cada parte exterior tenga su propio comportamiento.

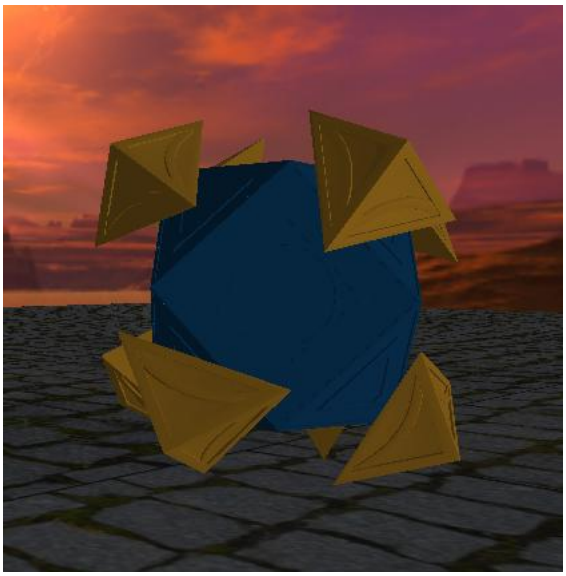


Figura 3. Resultado de la animación del Holocron durante la apertura y rotación de sus piezas exteriores.

Ejercicio 2. Despegue y trayectoria de un cohete espacial

Para el segundo ejercicio se agregó un modelo tridimensional de cohete junto con su textura

Además, se creó un shader independiente para representar el fuego producido por el motor.

Control del tiempo

El movimiento del cohete depende principalmente de:

```
double tiempoCohete = 0.0;
```

Durante cada fotograma se incrementa utilizando deltaTime.

```
tiempoCohete += deltaTimeRover;
```

También se implementó la tecla F11 para repetir el despegue.

```
if (reinicioCohete &&!reinicioCoheteAnterior)
{
    tiempoCohete = 0.0;
    humoCohete.clear();
}
```

Con esto se reinicia tanto el movimiento como las partículas generadas anteriormente.

Ecuaciones del despegue

La variable utilizada para calcular el movimiento se obtiene mediante:

```
const GLfloat tCohete =glm::clamp(tiempoCohete - 2.0,0.0,40.0);
```

Esto provoca que exista una pequeña espera antes de iniciar el movimiento.

La altura se calcula con una ecuación cuadrática:

```
-2.0f +0.8f *tCohete *tCohete
```

Por lo tanto, de manera simplificada:

$Y = -2 + 0.8t^2$

Al principio el movimiento es pequeño, pero conforme aumenta el tiempo la altura crece cada vez más rápido.

Para modificar progresivamente la trayectoria también se calcula:

const GLfloat curvaCohete =glm::max(tCohete - 3.0f,0.0f);

El desplazamiento lateral utiliza:

-60.0f -0.3f *curvaCohete *curvaCohete

Por lo tanto, la trayectoria puede resumirse como:

- X = -60 - 0.3(curva)²
- Y = -2 + 0.8t²
- Z = 35

De esta manera, el cohete comienza ascendiendo y posteriormente comienza a desplazarse lateralmente.

Inclinación del cohete

Además del cambio de posición, se calcula la orientación del modelo:

const GLfloat inclinacionCohete =tCohete > 0 ? std::atan2(0.6f * curvaCohete,1.6f * tCohete): 0;

Después se aplica:

padreCohete = glm::rotate(padreCohete,inclinacionCohete, glm::vec3(0,0,1));

Esto permite que el modelo vaya inclinándose conforme la trayectoria comienza a curvarse.

```
887         // F11 repite el despegue; y = y0 + 0.8*t*t
888         tiempoCohete += deltaTimeRover;
889         const bool reinicioCohete = mainWindow.getsKeys()[GLFW_KEY_F11];
890         if (reinicioCohete && !reinicioCoheteAnterior)
891         {
892             tiempoCohete = 0.0;
893             humoCohete.clear();
894             siguienteHumoCohete = tiempoHumo;
895         }
896         reinicioCoheteAnterior = reinicioCohete;
897         const GLfloat tCohete = static_cast<GLfloat>(glm::clamp(tiempoCohete-2.0,0.0,40.0));
898         const GLfloat curvaCohete = glm::max(tCohete-3.0f,0.0f);
899         const GLfloat inclinacionCohete = tCohete>0 ? std::atan2(0.6f*curvaCohete,1.6f*tCohete) : 0;
900         glm::mat4 padreCohete = glm::translate(glm::mat4(1),glm::vec3(
901             -60.0f-0.3f*curvaCohete*curvaCohete,-2.0f+0.8f*tCohete*tCohete,35.0f));
902         padreCohete = glm::rotate(padreCohete,inclinacionCohete,glm::vec3(0,0,1));
903         padreCohete = glm::rotate(padreCohete,glm::radians(25.0f),glm::vec3(0,1,0));
904         const bool motorCohete = tiempoCohete>=1.5 && tiempoCohete<42.0;
```

Figura 4. Cálculo de la posición e inclinación del cohete mediante ecuaciones dependientes del tiempo.

Fuego y humo del motor

El motor se considera activo cuando:

const bool motorCohete =tiempoCohete >= 1.5 && tiempoCohete < 42.0;

Esto permite que el efecto visual comience poco antes de que el cohete empiece a elevarse. El humo se genera calculando la posición que tenía el cohete en cada instante, después se modifica esta posición para colocar las partículas aproximadamente en la salida del motor.

Las partículas se almacenan en humoCohete y conservan el sistema de duración utilizado anteriormente.

Fuego del motor

También se agregó una llama independiente.

Su longitud varía ligeramente con el tiempo, esta variación evita que el fuego se observe completamente estático.

Para darle mayor volumen se dibujan tres planos cruzados debajo del motor, cada uno rotado 60 grados respecto al anterior.

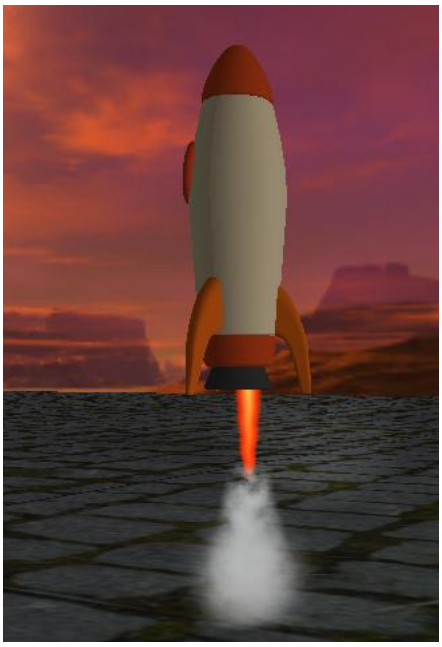


Figura 5. Inicio del despegue del cohete con activación del motor y generación de fuego y humo.

Trayectoria completa

A medida que aumenta el tiempo, la ecuación cuadrática incrementa rápidamente la altura del cohete.

Al mismo tiempo, curvaCohete modifica progresivamente la coordenada X y el ángulo calculado mediante atan2 cambia la inclinación del modelo.

Por esta razón el cohete no solamente asciende de forma vertical, sino que comienza a desviarse e inclinarse durante el vuelo.

La misma trayectoria se utiliza para calcular las posiciones donde deben aparecer las partículas de humo, logrando que el rastro permanezca relacionado con el recorrido del cohete.



Figura 6. Cohete espacial siguiendo automáticamente su trayectoria ascendente y curva, acompañado por los efectos de fuego y humo.

Resultados

En el primer ejercicio se logró reproducir una animación automática del Holocron utilizando las diferentes piezas que componen el modelo, el objeto se eleva progresivamente, permanece flotando mediante una pequeña oscilación vertical y gira continuamente alrededor de su eje.

También se implementó un ciclo para las ocho piezas exteriores. Después de permanecer cerrado durante los primeros segundos, las esquinas comienzan a separarse y rotar utilizando movimientos suavizados, las piezas superiores e inferiores giran en sentidos contrarios y finalmente regresan a su posición inicial, permitiendo repetir la animación continuamente.

En el segundo ejercicio se logró implementar un despegue automático del cohete mediante ecuaciones dependientes del tiempo. Durante la primera parte del movimiento aumenta principalmente su altura y posteriormente comienza un desplazamiento lateral progresivo, la orientación del cohete también cambia de acuerdo con la trayectoria, evitando que permanezca completamente vertical durante todo el vuelo y se agregaron efectos de fuego y humo al motor. La llama cambia ligeramente de tamaño con el tiempo y las partículas de humo se generan en las posiciones por donde pasa el cohete, haciendo más visible su trayectoria.

Problemas presentados

Una de las principales dificultades del Holocron fue animar las ocho piezas exteriores sin perder la relación con el cuerpo principal, para solucionarlo se utilizó una matriz padre para el movimiento general y posteriormente se aplicaron transformaciones independientes a cada esquina utilizando sus propios centros y ejes.

Otro problema fue evitar cambios bruscos durante la apertura y cierre, para ello se utilizó una función de suavizado, logrando que las piezas aceleren y desaceleren progresivamente.

En el caso del cohete fue necesario combinar diferentes ecuaciones para que el movimiento no fuera únicamente vertical, se utilizó una ecuación cuadrática para el ascenso y otra variación cuadrática para producir posteriormente el desplazamiento lateral, también fue necesario calcular una inclinación relacionada con la trayectoria para que la orientación visual del cohete coincidiera mejor con su movimiento.

Conclusión:

a) Complejidad y explicación: La principal complejidad fue coordinar varias transformaciones dependientes del tiempo, en el Holocron se combinaron la elevación, flotación, rotación y movimiento individual de ocho piezas manteniendo todo unido mediante una matriz padre, en el cohete fue necesario calcular simultáneamente su posición, inclinación, fuego y generación de humo a partir del tiempo, logrando que todos los elementos se desplazaran de forma coherente durante el despegue.
b) Comentarios generales: Estos ejercicios permitieron trabajar con animaciones más completas y automáticas, el Holocron mostró cómo un modelo dividido en varias piezas puede reproducir una secuencia compleja utilizando diferentes transformaciones, mientras que el cohete permitió utilizar ecuaciones para definir una trayectoria que cambia progresivamente con el tiempo, también fue útil reutilizar sistemas desarrollados anteriormente, como las partículas de humo, para adaptarlos a un nuevo objeto.
c) Conclusión: Se concluye que una animación compleja puede obtenerse combinando diferentes transformaciones y utilizando el tiempo como variable principal, en el caso del Holocron fue posible reproducir una secuencia de flotación, rotación, apertura y cierre mediante una matriz general y transformaciones

independientes para cada una de sus piezas, también se comprobó que las funciones matemáticas pueden utilizarse para definir trayectorias automáticas, el cohete utiliza ecuaciones cuadráticas para aumentar su altura y modificar progresivamente su desplazamiento lateral, mientras que su inclinación se calcula de acuerdo con el cambio de dirección del recorrido. Finalmente, la incorporación de humo y fuego permitió complementar la animación del despegue y hacer más evidente el funcionamiento del motor. Con esto se combinaron modelos formados por varias piezas, ecuaciones, transformaciones, shaders y sistemas de partículas dentro de una misma escena.

Bibliografía

- OpenGL-Tutorial.org. (s. f.). *Tutorial 3: Matrices*. OpenGL-Tutorial.org. https://www.opengl-tutorial.org/es/beginners-tutorials/tutorial-3-matrices/
- Martínez Ortiz, F. R. (2019). *Funciones trigonométricas: noción de amplitud, periodo y frecuencia*. Docencia Digital, Universidad Nacional Autónoma de México. https://www.docenciadigital.unam.mx/material/14854/funciones-trigonometricas-nocion-de-amplitud-periodo-y
- Godot Engine. (s. f.). *Sistemas de partículas (3D)*. Documentación de Godot Engine. https://docs.godotengine.org/es/4.x/tutorials/3d/particles/index.html
- Mozilla. (s. f.). *Tutorial de WebGL*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/API/WebGL_API/Tutorial
- Godot Engine. (s. f.). *3D*. Documentación de Godot Engine. https://docs.godotengine.org/es/4.5/tutorials/3d/